

Software Engineering – Modeling & UML

(Exam-Oriented Complete Notes with Intuitive Explanations)

1. Why We Model?

Simple Idea

Modeling means **creating a simplified representation** of a real system so that we can **understand, analyze, and build it better**.

Why is Modeling Important?

- Modeling is a **proven engineering technique** used in all successful engineering fields (buildings, machines, roads).
- Failed software projects fail in different ways, but **successful projects share common practices** — one major one is **modeling**.
- Modeling helps us **understand the system before building it**.

Key Purpose

We model **to understand the system we are developing** before coding it.

2. What is a Model?

Definition

- A **model is a simplification of reality**.
- A model provides **blueprints** of a system.

Intuitive Example

- A **map** is a model of a city.
- A **building blueprint** is a model of a house.
- A **UML diagram** is a model of a software system.

Models remove unnecessary details and keep only what is important.

3. Benefits of Modeling

1. Visualization

- Helps us **see** the system as it exists or as we want it to be.

2. Specification

- Models specify the **structure** and **behavior** of the system.

3. Construction Guide

- Models act as a **template or guide** for developers while building the system.

4. Documentation

- Models document **design decisions** for future reference.
-

4. Limitation of Human Ability

Problem

- Humans **cannot understand very large and complex systems all at once.**

Why Modeling Helps

- Complex systems are broken into **smaller subsystems.**
- We focus on **one aspect at a time.**

Example

- Human body is complex → studied as heart, brain, lungs, etc.
 - Software system → divided into modules, components, diagrams.
-

5. Principles of Modeling

Principle 1: Choose Models Carefully

- The choice of models **shapes the solution.**
- Right models help solve **complex (wicked) problems.**

Example: Roads vs Buildings require different models.

Principle 2: Different Levels of Precision

- Models can be **simple or detailed**.
- Depends on:
 - Who is viewing the model
 - Why they are viewing it

Example

- Customer → simple UI mockups (e.g., Balsamiq)
 - Developer → detailed technical diagrams
-

Principle 3: Models Must Be Connected to Reality

- A model should reflect the **real-world system**.
 - Too abstract models are **hard to understand**.
- Closer to reality = easier to understand and explain.
-

Principle 4: No Single Model is Enough

- Complex systems need **multiple models**.
- Each model shows a **different view** of the system.

Example

- Human body → skeletal, muscular, nervous systems
 - Software → use case, class, sequence diagrams
-

6. Requirement Modeling

What is Requirement Modeling?

- First technical step in software engineering.
- Converts user needs into **formal representations**.

Output

- A **set of models** describing:
 - What the system should do
 - How users interact with it

Requirements model is the foundation of design and development.

7. Scenario-Based Modeling

Why Scenario-Based Modeling?

- User satisfaction is the **top success factor**.
- Understanding user interaction leads to **better requirements**.

Key Idea

- Describe the system from the **user's point of view**.
- Use **scenarios** showing how users interact with the system.

UML Approach

- Scenarios are captured using **Use Case Diagrams**.
-

8. UML (Unified Modeling Language)

Definition

UML is a **standardized general-purpose modeling language** for object-oriented software engineering.

Purpose

- Create **visual models** of software systems.
 - Provide a **common language** for developers, designers, and stakeholders.
-

9. UML as a Language for Visualizing

Key Features

- Independent of programming language
- Can be understood by **humans and tools**
- Can be mapped to:
 - C++
 - Java
 - C#
 - Databases (RDBMS)

One UML model can be interpreted consistently everywhere.

10. Goals of UML

1. **Visibility** – Easy to see and understand system structure
 2. **Extensibility** – Can be extended for special needs
 3. **Independence** – Not tied to any programming language
 4. **Object-Oriented** – Supports OO concepts
-

11. Use Case Modeling

When Does It Start?

- After:
- Recognition of need
- Feasibility study

How Requirements Are Gathered

- Interviews
- Questionnaires
- Observing existing systems

Purpose of Use Cases

- Capture **functional requirements**
 - Describe **what the system does**, not how
 - Used for **testing (white-box testing)**
-

12. Actors and Use Cases

Actors

- External entities interacting with the system
- Can be:
- Humans
- Devices
- Other systems

Actor = Anything outside the system that communicates with it

Use Cases

- Actions performed by actors
 - Represent system functionalities
-

13. Use Case Relationships

1. Association

- Simple connection between actor and use case

2. Include Relationship

- One use case **always includes** another
- Used to avoid repetition

3. Extend Relationship

- Optional or conditional behavior
- Base use case can work without extension

4. Generalization Relationship

- Inheritance between actors or use cases
 - Child inherits behavior from parent
-

14. Steps to Draw a Use Case Diagram

1. Identify actors
 2. Identify use cases
 3. Connect actors and use cases
 4. Draw system boundary
 5. Define relationships
 6. Review and refine
 7. Validate with stakeholders
-

15. Example: Online Shopping System

Actors

- Customer
- Admin

Use Cases

- Browse Products
- Add to Cart
- Checkout
- Manage Inventory (Admin)

Requirements

- Customer can browse, add, and checkout
 - Admin manages inventory
-

16. Use Case Diagram Tools

- draw.io
 - Microsoft Visio
-

17. Case Study: Career Guidance Portal

Typical Actors

- Student
- Counselor
- Admin

Typical Use Cases

- Register/Login
 - View Career Options
 - Take Aptitude Test
 - Get Counseling
-

Final Exam Tip

- Always define terms clearly
 - Use examples and diagrams
 - Write step-by-step explanations
 - Relate concepts to real-life scenarios
-

End of Module – Fully Covered for Final Exam Preparation